第一周实验逐项说明

读者：吴源泰 · **目的**：把 Week 1（Phase 0）每一步讲清楚——做了什么、怎么做的、看哪个文件、举一个具体例子

时间：2026-10-07 ~ 2026-10-10 · **项目根**：`F:\research` · **主机**：`laptop-csp8hpp8`

> *配套文件：`week1\W1_EVIDENCE.md`（任务清单式证据）、`week1\environment.md`（环境清单）、`week1\W1_READING_NOTES.md`（阅读提纲与三角不等式例子）、`week1\W1_WEEKLY_REPORT.pdf`（一页周报）。*
>
> *本文是**教学向**的：按"我遇到什么 → 我怎么想 → 我怎么做 → 结果是什么"的顺序讲。*

0. 先回答：为什么 `run.py` 没出图

```
python scripts\run.py          # 等价于 run.py list —— 只打印任务表（你截图看到的）
python scripts\run.py figures  # 这才是出图
```

出图结果在 `F:\research\reports\figures\`，每个图有 `.pdf`（矢量，给论文）和 `.png`（快速看）两种格式，外加一个 `manifest.json` 记录每个文件的 sha256。

1. 这一周到底要解决什么问题

Phase 0 的唯一目标：**让"环境"这件事变得可信**。听起来像杂活，但后面所有数字——分歧率、升级率、时延、内存——都挂在这上面。如果环境说不清，论文里每个数字都会被质疑。

导师 handout 把 Phase 0 的成功判据写得很清楚：

> *另一次运行能解析出同一环境、同一划分与有限的分数；**实际设备上限被写下来**。*

所以这一周要留下四类证据：**① 环境锁定 ② 源码 commit ③ 数据划分 ④ 一次实测计时**。下面每一步都对应其中一项。

2. 第 1 步：先把机器摸清楚（A14-1：硬件/数据清单）

做了什么

按 handout 要求记录 CPU、GPU、显存、内存、空闲磁盘、可用设备、数据可获取性。

怎么做的（含一次绕路）

第一反应是用系统的 WMI 接口：

```
Get-CimInstance Win32_OperatingSystem     # 期望拿到内存总量
Get-CimInstance Win32_VideoController     # 期望拿到显存
```

结果：`拒绝访问`。 受限环境下 WMI 被拒绝。于是改换两条通道：

想拿的信息	换用的方法	命令
操作系统版本	读注册表	`Get-ItemProperty 'HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion'`
CPU 型号	读注册表	`(Get-ItemProperty 'HKLM:\HARDWARE\DESCRIPTION\System\CentralProcessor\0').ProcessorNameString`
内存总量	Node 的 os 模块	`node -e "console.log(require('os').totalmem()/2**30)"`
显卡/显存	NVIDIA 自带工具	`nvidia-smi --query-gpu=name,memory.total,memory.free,compute_cap --format=csv`
磁盘空闲	.NET 的 DriveInfo	`[System.IO.DriveInfo]::GetDrives()`

结果（实测值）

项	值	备注
系统	Windows 11 25H2，build 26200.9457	⚠ 注册表里 `ProductName` 仍写 "Windows 10 Home China" —— Win11 的遗留字段，**不能直接抄**
CPU / 内存	i7-14700HX（28 逻辑核）/ **31.8 GB**	
GPU / 显存	RTX 4070 Laptop / **8187.5 MiB**	驱动 591.74，算力 **sm_89**，驱动报告的 CUDA 上限 13.1
功耗档	Current / Requested = **80 W**（Default 55 W）	跑时延必须绑定这个档位，否则数字不可比
磁盘	C 116.7 / F **160.2** / G **522.5** GB 空闲	项目在 F:，G: 备用

例子：为什么"不猜"很重要

如果我不去读注册表、而是凭常识写"Windows 11"，`environment.md` 里就没有那条 ⚠ 记录；而恰恰是这个字段在同事复核时会引发"你到底在什么系统上跑的"争论。**凡是能从机器读出来的，绝不凭记忆写。**

参考文件：`week1\environment.md` 第 1 节 · **复现命令**见该文件第 8 节。

3. 第 2 步：环境隔离（Phase 0 的核心）

为什么要隔离——不是洁癖，是硬冲突

两个官方仓库各自钉死了互相矛盾的依赖：

- AnomalyDINO 的 `requirements.txt` 原文：`numpy==1.26 # numpy 2.0 not compatible at the moment`
- PatchCore 侧实测 numpy 2.2.6 正常

这两个 pin 不可能同时满足。

怎么做的（一小步一小步）

① 先看 conda 把环境放哪（这一步救了后面的时间）：

```
conda config --show envs_dirs
# 输出第一项是 C:\Users\yuantai\.conda\envs  —— 不是 E:\anaconda\envs
```

后果：`conda create -n X` 建在 **C 盘**，而 `check_env.ps1` 默认去 `E:\anaconda\envs` 找 → 找不到。

记下来：所有脚本都要用绝对路径 `-Python C:\Users\yuantai\.conda\envs\...\python.exe`。

② 建第一个环境并装 torch：

```
conda create -n adedge-patchcore python=3.10 -y
python -m pip install torch torchvision --index-url https://download.pytorch.org/whl/cu124
```

为什么必须是 **cu124**：你的 GPU 是 sm_89（Ada），**只有 CUDA ≥ 11.8 的构建才带 sm_89 的 kernel**，即 torch ≥ 2.0 的 cu118+。老仓库原始 pin 的 cu111 wheel 装上去，第一次卷积就会报

```
RuntimeError: CUDA error: no kernel image is available for execution on the device
```

③ 试跑官方 CLI，暴露两个依赖缺口：

```
$env:PYTHONPATH = 'F:\research\repos\patchcore-inspection\src'
python bin\run_patchcore.py --help
# ModuleNotFoundError: No module named 'timm'     ← requirements.txt 里没写！
python run_anomalydino.py --help
# ModuleNotFoundError: No module named 'yaml'     ← 需要 PyYAML
```

补装：`pip install timm pyyaml`。**这两个是官方仓库自己的依赖缺口，值得记进报告。**

④ 建第二个环境并降 numpy：

```
conda create -n adedge-anomalydino --clone adedge-patchcore -y   # 本地复制，不重新下载
python -m pip install "numpy==1.26.4"
```

⑤ 立刻被 `pip check` 打回来：

```
opencv-python 5.0.0.93 requires numpy>=2, but you have numpy 1.26.4 which is incompatible.
```

试着小步修：把 opencv 降到 4.14 → **pip 又把 numpy 拉回 2.2.6**。

最终结论：**必须同时钉住两个包：**

```
python -m pip install "opencv-python==4.10.0.84" "numpy==1.26.4"
```

结果

环境	python	numpy	opencv	torch	pip check	freeze sha256
adedge-patchcore	3.10.22	2.2.6	5.0.0	2.6.0+cu124	No broken requirements	70612E24…DAB6
adedge-anomalydino	3.10.22	**1.26.4**	**4.10.0.84**	2.6.0+cu124	No broken requirements	2E1C8823…9A8E

同一台机器、同一个 PyTorch，两套互斥的 numpy/opencv 组合各自干净共存 —— 这就是"隔离"的实证。

参考文件：logs\pip_freeze_*.txt、logs\env_*.json、scripts\check_env.ps1

4. 第 3 步：目录骨架（东西放哪、谁是只读的）

目录	放什么	读写规则
repos\	官方仓库源码（PatchCore、AnomalyDINO）	**只读**。要改代码就导出补丁到 project\patches\
data\	数据集	**只读**。下载后不再改动
cache\	权重、特征缓存（torch\ hf\ weights\ features\）	可写，可重建
manifests\	reference-fit / development / calibration 清单 CSV	改一行＝换了一份数据集，必须重记哈希
configs\	运行配置	可写
runs\	每次实验一个子目录（含 run.json）	只增不改
logs\	锚点报告、环境报告、各项运行的原始输出	只增不改（**这是证据**）
scripts\	PowerShell / Python 工具脚本	可写，进版本控制
reports\	图表、周报	可写，进版本控制
archive\	上游源码归档（含 sha256 记录）	只增不改
project\src\	自己写的代码	可写，进版本控制
tmp\	临时文件（合成数据、渲染产物）	可清空，不进版本控制

关键约定：data\ 与 repos\ 绝不写入。理由很实际——一个脚本不小心改写了数据集，

你后面所有分数都失去意义，而且**无法察觉**。

5. 第 4 步：仓库锚定（含"git clone 坏了"这件事）

目标

回答一个问题：**凭什么说这次运行可复现？** 答案是"代码+权重+数据都有不可变标识"。

怎么做的

① 先试常规做法，失败了：

```
git clone https://github.com/amazon-science/patchcore-inspection.git
# 5 分钟无输出，卡在 pack 传输阶段
```

依次排除：--depth 1、http.version=HTTP/1.1、credential.helper=、GIT_TERMINAL_PROMPT=0（禁用交互提示）

—— **五种组合全部卡死**。

② **判断问题在哪**：

```
git ls-remote https://github.com/amazon-science/patchcore-inspection.git
# 1 秒返回：fcaa92f124fb1ad74a7acf56726decd4b27cbcad  refs/heads/main
```

`ls-remote` 正常说明**网络和认证都没问题**，坏的是 pack 传输。于是换通道：

用 `Invoke-WebRequest` 走 .NET 的 schannel 下载 codeload 的 ZIP —— **2.3 MB 只用 1–2 秒**。

③ **把 commit 钉进 URL**（这样就不依赖分支名了）：

```
https://codeload.github.com/amazon-science/patchcore-inspection/zip/fcaa92f124fb1ad74a7acf56726decd4b27cbcad
```

④ **记录四样东西，其中"树指纹"最关键**：

记录项	例子
来源 URL	`https://github.com/amazon-science/patchcore-inspection`
commit	`fcaa92f124fb1ad74a7acf56726decd4b27cbcad`（2023-03-06）
归档 sha256	`7FDBA8B2B1D3586B…39F072`
树指纹	`419fbc575b1bfac2…be7d27`

树指纹怎么算：把解压后每个文件求 sha256，拼成 `相对路径|文件哈希` 的列表，排序后整体再求一次 sha256。

它比 commit 号更硬——因为它校验的是**你磁盘上实际躺着的那些字节**。

结果

名称	commit	文件数	树指纹（前缀）
patchcore-inspection	`fcaa92f1…`（2023-03-06，3.6 年未更新）	401	`419fbc57…`
AnomalyDINO	`b9d1c264…`（2025-12-17）	14	`d23f5352…`
mvtec_ad_evaluation（官方评测码，公开直链）	—	11	`74059be4…`

副作用（必须记住）

ZIP 快照**没有 git 历史**，所以"改代码后导出补丁"这条老路走不通。替代方案：

```
git diff --no-index 原文件 改后文件 > project\patches\xxx.patch
```

`--no-index` 不需要仓库，专门用来比两个普通文件。

参考文件：`scripts\fetch_sources.ps1`、`logs\anchors_latest.json`、`archive\sources\`

6. 第 5 步：环境验收——为什么"装上了"不等于"能跑"

一个真实的坑

`torch.cuda.is_available()` 返回 `True` **并不保证能算**：如果 wheel 里没有为 sm_89 编译的 kernel，

它会一直返回 True，直到你第一次做卷积才崩，报 `no kernel image is available`。

所以验收必须**真跑一次前向+反向**：

```
x = torch.randn(8, 3, 256, 256, device="cuda")
conv = torch.nn.Conv2d(3, 16, 3, padding=1).to("cuda")
y = conv(x); y.mean().backward(); torch.cuda.synchronize()
```

结果

```
GPU 冒烟    : OK  (峰值 70.0 MiB)
torch       : 2.6.0+cu124  (cuda runtime 12.4, cudnn 90100)
device      : NVIDIA GeForce RTX 4070 Laptop GPU  sm_89  8187.5 MiB
freeze      : pip_freeze_adedge-patchcore_20261010-102504.txt  sha256 70612E24…DAB6
```

为什么要导出 freeze：`environment.md` 里的版本表是给人看的；`pip freeze` 的 sha256 是给机器比的。

以后每次运行的 `run.json` 都要记这个哈希——**版本表可以抄错，哈希不会**。

参考文件：`scripts\check_env.ps1`、`logs\env_adedge-patchcore_20261010-102504.json`

7. 第 6 步：显存上限——一次完整的"测法错误"记录

这一条是本周最有价值的方法学发现，值得完整讲。

① 第一版测法（错的）

思路很自然：**"从大往小试 batch，不 OOM 的最后一个就是上限"**。

跑出来：512×512 分辨率下 batch 192 通过，峰值显存 `max_memory_allocated() = 10835 MiB`。

② 发现不对

你的显卡只有 8187 MiB，怎么可能分配 10835 MiB？

③ 找到原因

Windows 驱动允许**显存静默溢出到主机内存**（shared GPU memory）。

它**不报 OOM**，只是把数据在显存和内存之间搬来搬去 —— 所以：

512×512	耗时
batch 256（峰值 14355 MiB，175% 物理显存）	**71.7 s**
batch 128（峰值 7315 MiB，89%）	**3.7 s**

慢 20 倍，但一点错都不报。 如果我按第一版测法设定实验 batch，第 7 周会得到一组

"看起来能跑、实际在主机内存里跑"的时延数字，**而且从表面上完全看不出来**。

④ 改成双判据

- **安全上限** = 峰值 ≤ 80% 物理显存（6550 MiB）
- **溢出警戒** = 峰值 > 100% 物理显存

再补一次从大到小的完整扫描，得到：

分辨率	安全 batch	峰值	占物理显存	溢出起点
256×256	256	3787 MiB	46.3%	未观测到
384×384	192	6215 MiB	75.9%	256（100.1%）
512×512	112	6435 MiB	78.6%	160（110.8%）

（骨干 `wide_resnet50_2` 冻结，权重常驻 265.1 MiB；**两轮扫描结果一致**。）

⑤ 用线性模型交叉验证

峰值随 batch 线性增长：`峰值 ≈ 常驻 + k × batch`。实测 k 分别为 13.8 / 31.0 / 55.0 MiB 每张。

预测 vs 实测：384px 预测 202 vs 实测 ≥192 ✓；512px 预测 114 vs 实测 112 ✓。**模型站得住**。

参考文件：`project\src\w1_vram_limit.py`、`logs\w1_vram_limit.json`、`reports\figures\figure_1_vram_limits.png`

8. 第 7 步：A14-3 手算边界（本周的核心数学动作）

题目

6 个参考向量、2 个观测中心，算出 `L(q) ≤ s_M(q) ≤ U(q)`，并包含"阈值并列"和"需要升级"两种情形。

我构造的例子

```
M = [ (0,0), (1,0), (0,1), (2,0), (10,0), (10,0) ]     ← 最后两个完全相同（重复向量）
观测中心：c0 = m0 = (0,0),   c1 = m4 = (10,0)          ← 中心本身也是参考点
阈值 t = 1.5      报警定义： alarm ⇔ s_M(q) > t        ← 严格大于
```

一步步算

第一步：分 cell。 每个参考点归到离它最近的中心。

参考点	到 c0 距离	到 c1 距离	归入
(0,0)	0	10	cell0
(1,0)	1	9	cell0
(0,1)	1	10.05	cell0
(2,0)	2	8	cell0
(10,0)	10	0	cell1
(10,0)	10	0	cell1

第二步：算保守半径 $r_j = \max_{m \in C_j} d(m, c_j)$：

- cell0 = {(0,0),(1,0),(0,1),(2,0)} → r0 = max(0, 1, 1, 2) = **2**
- cell1 = {(10,0),(10,0)} → r1 = max(0, 0) = **0**（两个成员重合，半径退化为一点）

第三步：对每个查询点算三个数

- $s_M(q)$ = 到**全部 6 个**参考点的最近距离（真值）
- $L(q) = \min_j \max\{0, d(q,c_j) - r_j\}$（下界）
- $U(q) = \min_j d(q,c_j)$（上界）

查询	坐标	s_M	L	U	完整库真值	筛查结论	覆盖情形
q1	(0.5, 0)	0.500	0	0.500	normal	**本地判 normal**	$U \le t$
q2	(0, 10)	9.000	8.000	10.000	alarm	**本地判 alarm**	$L > t$
q3	(3, 0)	1.000	1.000	3.000	normal	升级 → normal	区间跨阈值
q4	(0, 3)	2.000	1.000	3.000	**alarm**	升级 → alarm	区间跨阈值
q5	(2, 1.5)	**1.500**	0.500	2.500	normal	升级 → normal	$s_M = t$（并列）

核对结果：区间违规数 **0**；本地已解决项与真值不一致 **0**；本地解决比例 2/5 = 40%。

三角不等式的平实解释

三角形任意一边都不超过另外两边之和。

已知参考点 m 与中心 c（都在集合里），对任意 q：

```
d(q,c) ≤ d(q,m) + d(m,c)        ⇒        d(q,c) − d(m,c) ≤ d(q,m)
```

因为 cell 里所有 m 都满足 `d(m,c) ≤ r`，所以 **`d(q,c) − r` 是"q 到本 cell 任意参考点距离"的下界**。

而中心 c 自己就是参考点，所以 `d(q,c)` 是**上界**。对所有 cell 取下确界，就得到 `L ≤ s ≤ U`。

q4 为什么最重要

q4 **(0,3)** 在 cell0 的覆盖球**外面**，但它的真值是 **alarm**（s=2 > t=1.5）。

如果有人图省事用"离中心足够近就当正常"这种启发式，q4 就会被误判。

区间必须整体落在阈值一侧才能下结论 —— 这正是"保守筛查"和"拍脑袋阈值"的区别。

参考文件：`project\src\w1_bounds_example.py`（可重跑）、`reports\figures\figure_2_bounds_geometry.png`（几何图）、`figure_3_decision_map.png`（黄色＝升级区）

9. 第 8 步：A14-2 距离检查（数据待定时的替代路径）+ 一次更正

为什么走这条路

A14 原文明确允许：

If data access is pending, run tensor-distance checks on ten artificial 2-D vectors.

数据集未获批 → 走这条被许可的替代路径，同时把 A6 要求的数值契约固定下来。

Part A：十个人造二维向量

朴素双重循环 vs 向量化，4 个查询逐个核对 → **最大绝对差 0.00e+00**。

Part B：分块 vs 暴力精确最近邻

B0 先算一笔账：整块物化 `(P=512, N=20000, D=384)` 的 float64 距离矩阵需要

`512 × 20000 × 384 × 8 字节 = 29.3 GiB` → **本机 32 GB 内存下不可行**。

所以分块不是优化，是必需。

B1 小规模验证：整块 0.194 s vs 分块 0.194 s，最大绝对差 0.00e+00 ✓

B2 真实规模 → 这里出了一次事故：

我最初写死了 `bs=256`，单块中间张量 `= 256 × 20000 × 384 × 8 = 14.6 GiB`。

第一次跑的时候内存恰好空闲，**跑通了，还得出"bs=32 比 bs=256 快 3.5 倍"的结论**；

第二次同一条命令直接崩：

```
numpy._core._exceptions._ArrayMemoryError: Unable to allocate 14.6 GiB
```

结论作废，改为按内存预算反推块大小（预算 2 GiB → 上限 34）。修正后：

块大小	单块内存	用时	与 bs=34 的差
bs=34	1.95 GiB	16.58 s	0
bs=32	1.83 GiB	16.71 s	0
bs=16	0.92 GiB	16.66 s	0

块大小	单块内存	用时	与 bs=34 的差
bs=8	0.46 GiB	16.75 s	0

原来那"3.5 倍差距"消失了 —— 说明它本来就是内存压力伪影。

教训：一次跑通 ≠ 可复现。凡是"靠当时机器状态才成立"的数字，都不能进结论。

Part C：GPU vs CPU（以及一个必须记住的数值细节）

- CPU 分块 0.0245 s vs GPU 分块 0.0077 s（库已在显存内，**不含编码器前向**）
- **GPU(float32) 与 CPU 的最近距离最大绝对差 = 2.38e-07**

最后这个数字很关键：我在距离核对里固定的容忍值 `TOL = 1e-9` 是针对 **float64/numpy** 的，

而 **GPU float32 的路径差异比它大两个数量级**。第 4–6 周设计区间保护量时，

如果直接沿用 1e-9，会把正常的浮点噪声误判成"区间违规"。**保护量必须按实际计算路径取。**

（另外：GPU 加速比两次跑出 7.4× 和 3.2×，**随机器负载波动**，第 7 周报时延必须在固定功耗档、空载下重测。）

参考文件：`project\src\w1_distance_checks.py`、`logs\w1_distance_checks.log`

10. 第 9 步：A14-4 三种角色清单

为什么要分三种角色

阈值 `t` 必须在**模型没见过的正常样本**上定，否则就是自己给自己出题。所以正常样本要拆成三份互不重叠的角色：

角色	用途	建议规模
reference-fit	只用来构建参考库（1/4/16-shot 的抽样池）	16 张
development	调参、试预算、看中间曲线	24 张
calibration	**只用来定阈值 t**（5% 名义误报率的分位数）	其余全部

关键纪律：同一张原图绝不跨角色；测试集的缺陷标签在协议冻结前一概不读。

怎么保证不泄漏（两个技术点）

1. **只扫 `train/good`**（AD 2 再加 `validation/good`）。脚本**根本不读 `test/` 的内容**，只统计文件个数。

2. **确定性划分：**按 `sha256(文件名 + seed)` 排序切片，而不是用 `random.shuffle`。

原因：`random.shuffle` 的行为依赖列表顺序和 Python 版本，换台机器可能得到不同划分；

而"文件名+种子的哈希"在任何机器上都一样。

自检结果（用合成数据，因为真实数据还没到）

```
3 类 × 100 张正常样本 → reference_fit 16 / development 24 / calibration 60
跨角色共享同一 sha256 的样本数 : 0
跨角色共享同一路径的样本数     : 0
复性：两次运行的清逐字一致
自检结论：通过
```

真实模式现在的结果（预期如此，不是 bug）：

```
[MISS] bottle  找不到正常样本（F:\research\data\mvtec_ad\bottle\train\good）
[MISS] tile    ...
[MISS] screw   ...
-> 真实清单未生成。MVTec AD 需在官网填表获取（本机尚未获批）。
```

参考文件：`scripts\make_manifests.py`、`logs\w1_manifests_selftest.log`

11. 第 10 步：出图流水线

四条规矩（写死在脚本里）

1. **只从 JSON 出图**，不许用正则从文本日志里抓数字。文本日志给人看，JSON 给机器读；否则数据不可复核，图画错了也发现不了。
2. **确定性**：不用随机数、不写绝对路径、PDF 关掉 `CreationDate`。
3. **双格式**：`.pdf`（矢量，进论文）+ `.png`（200 dpi，快速看）+ `manifest.json`（逐个文件记 sha256）。
4. **路径可移植**：全部相对仓库根计算，所以克隆到任何机器都能重跑。

最容易踩的坑：中文字体

matplotlib 默认字体没有中文，标签会渲染成方框。脚本开头这两行不能删：

```
plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "SimSun", "DejaVu Sans"]
plt.rcParams["axes.unicode_minus"] = False
```

位级可复现的验证

- 同机跑两遍：**7 个文件（3 图 × 2 格式 + manifest）哈希完全一致**
- 从 GitHub 克隆到干净目录再跑：**7 个文件依旧完全一致**

（第一次验证时 `manifest.json` 不一致，原因是我在里面写了绝对路径 `repo_root`；已删除并复验通过。）

三张图分别说明什么

图	说明什么
`figure_1_vram_limits`	峰值显存随 batch 线性增长；黑实线＝物理显存，虚线＝80% 安全线；右下角框列出各分辨率安全 batch
`figure_2_bounds_geometry`	左：参考点、观测中心、覆盖球；右：5 个查询的 `[L,U]` 区间与阈值 t 的关系
`figure_3_decision_map`	蓝＝本地判正常，红＝本地判报警，**黄＝区间跨阈值必须升级查询**（该视野占 21%）——黄色越宽，省下的整库查询越少

参考文件：`scripts\make_figures.py`、`reports\figures\`

12. 统一入口：`scripts\run.py`

命令	干什么	输出
`run.py list`	列出所有任务（不带参数时的默认行为）	终端
`run.py env` / `env-dino`	环境验收：GPU 冒烟 + 分辨率上限 + freeze	`logs\env_*.json`
`run.py anchors`	重新锚定上游源码（ZIP + 树指纹）	`logs\anchors_latest.json`
`run.py bounds`	A14-3 手算例子	终端（0 违规）
`run.py distance`	距离检查 + 分块/暴力核对 + GPU 计时	终端（约 70 s）
`run.py vram`	显存安全上限实测	`logs\w1_vram_limit.json`
`run.py manifests`	三角色清单合成自检（不需数据）	`tmp\synthetic_manifests\*.csv`
`run.py manifests-real`	真实数据清单（需 MVTec AD）	`manifests\*.csv`
`run.py figures`	出图	`reports\figures\`
`run.py all`	bounds + distance + manifests + figures	上述各处

为什么需要它：这台机器有三个 Python，只有两个装了 torch，而 conda 环境不在默认路径。

`run.py` 会自动找到装了 torch 的解释器——**即使用默认 Python 启动它也会自己切过去**（已实测）。

13. 参考文件总表

文件	作用	关键内容	怎么查看
week1\environment.md	环境清单	硬件/软件/锚点/数据/未知项	直接读
week1\W1_EVIDENCE.md	任务证据清单	每项任务的状态、证据、边界	直接读
week1\W1_READING_NOTES.md	阅读提纲	三个三角不等式例子、R7/R1 要点、五个未决问题	直接读
week1\W1_WEEKLY_REPORT.pdf	一页周报	Done/Evidence/Blockers/Next	直接打开
logs\anchors_latest.json	源码锚点	commit、归档 sha256、树指纹	Get-Content -Raw logs\anchors_latest.json | ConvertFrom-Json
logs\env_*.json	环境实测	torch 版本、CUDA、显存、分辨率上限	同上
logs\pip_freeze_*.txt	依赖锁定	全部包版本	直接读；sha256 记进 run.json
logs\w1_vram_limit.json	显存扫描原始数据	每个 batch 的峰值与状态	同上（图 1 的数据源）
logs\w1_distance_checks*.log	距离检查原始输出	Part A/B/C 全部数字	直接读
logs\w1_manifests_selftest.log	清单自检	划分结果与互斥检查	直接读
scripts\run.py	统一入口	所有任务	python scripts\run.py list
scripts\check_env.ps1	环境验收	GPU 冒烟 + 分辨率扫描	run.py env
scripts\fetch_sources.ps1	源码锚定	ZIP + 树指纹	run.py anchors
scripts\make_manifests.py	角色清单	三角色划分与互斥复查	run.py manifests
scripts\make_figures.py	出图	三张图 + manifest	run.py figures
scripts\md2docx.py	文档转换	Markdown → DOCX	python scripts\md2docx.py 输入.md 输出.docx
project\src\w1_bounds_example.py	A14-3 手算	5 种边界情形	run.py bounds
project\src\w1_distance_checks.py	A14-2 距离检查	人造向量 + 分块 + GPU	run.py distance
project\src\w1_vram_limit.py	显存实测	安全上限 + 溢出警戒	run.py vram
reports\figures\	图表	3 图 × (pdf+png) + manifest	直接打开
archive\sources\	上游源码归档	3 个压缩包 + 哈希记录	直接读

14. 命令速查（复制即用）

```
# 看有哪些任务
cd F:\research ; python scripts\run.py list

# 出（你才想做的）
python scripts\run.py figures
start F:\research\reports\figures

# 跑一遍所有不需要数据的任务
python scripts\run.py all

# 环境验收（需要 GPU 空闲）
python scripts\run.py env

# 单独跑手算例子
python scripts\run.py bounds

# 想看某个原始证据
Get-Content -Raw F:\research\logs\anchors_latest.json | ConvertFrom-Json
```

15. 这一周没有做什么（边界）

- ❌ 没有复现 PatchCore / AnomalyDINO（那是 Week 2，且需要数据）
- ❌ 没有在真实 MVTec 图像上跑任何东西（数据未获批）
- ❌ 没有设计或实现筛查方法（Week 4）
- ❌ 没有碰 YOLO 剪枝 / 扩散增强 / 多模态（handout 明令试点问题验证前不许开始）

- ❌ 没有调任何阈值（连一个真实分数都还没有）
- ❌ 没有绕开 MVTec 的登记表去用第三方镜像
- ❌ 没有对油气现场做任何泛化声明

下周（等数据）：① 生成真实三角色清单并复查互斥；② 补做被阻塞的检查（列图像/标签、显示图像与掩码、跑一个 batch、PatchCore 最小示例、AnomalyDINO 1-shot 冒烟）；③ 补录权重 sha256 + 落地 `run.json` 模板。